

Complete end-to-end case study aligned with (5 days / 40 hours) and using **ASP.NET Core REST API + Angular, Clean Architecture, Microservices, EF Core Code First, and JWT security.**

Case Study Title

Mortgage Loan Management System (MLMS)

A simplified, enterprise-style mortgage application system

1. Business Overview (Problem Statement)

A bank wants to build a **Mortgage Loan Management System** to manage:

- Customer mortgage loan applications
- Loan approval workflow
- Loan status tracking
- Role-based access (Customer, Loan Officer, Admin)

The system must follow **modern enterprise architecture**:

- Frontend: Angular
 - Backend: ASP.NET Core REST APIs
 - Architecture: Clean Architecture + Microservices
 - Security: JWT-based authentication & authorization
-

2. Scope

In Scope

- Create and submit mortgage applications
- Approve or reject applications
- View loan status
- Role-based access control
- Basic validations
- RESTful APIs with EF Core Code First

Out of Scope

- Credit score integration
 - Payment schedules
 - Document uploads
 - External bank APIs
-

3. Roles (Who Uses the System)

Role	Description
Customer	Applies for mortgage loan, views status
Loan Officer	Reviews, approves, or rejects applications
Admin	Manages users and roles

4. Microservices Breakdown

Services to be Implemented (Limited & Manageable)

Microservice	Responsibility
Identity Service	Authentication, JWT, Roles
Customer Service	Customer profile management
Loan Service	Mortgage loan application lifecycle
API Gateway (Optional – Basic)	Single entry point

5. Use Cases per Service

5.1 Identity Service

Purpose: Authentication & Authorization

Use Cases

- Register user
- Login user
- Generate JWT token
- Assign role to user

Roles Involved

- Customer
 - Loan Officer
 - Admin
-

5.2 Customer Service

Purpose: Manage customer profile

Use Cases

- Create customer profile
- View customer details
- Update basic information

Simple Entity

- CustomerId
 - FullName
 - Email
 - Phone
-

5.3 Loan Service (Core Business Service)

Purpose: Mortgage loan processing

Use Cases

- Submit loan application
- View loan status

- Approve loan (Loan Officer)
- Reject loan (Loan Officer)

Loan Status

- Submitted
 - Approved
 - Rejected
-

6. Sample User Stories

Customer Stories

1. As a **Customer**, I want to apply for a mortgage loan.
2. As a **Customer**, I want to view my loan application status.

Loan Officer Stories

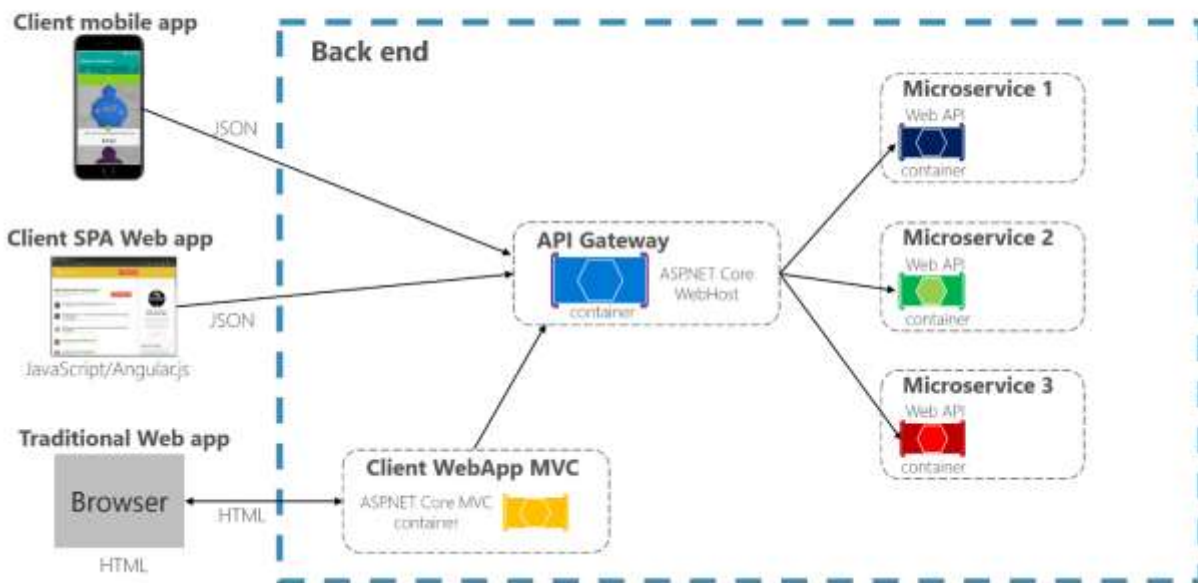
3. As a **Loan Officer**, I want to view all submitted loan applications.
4. As a **Loan Officer**, I want to approve or reject a loan.

Admin Stories

5. As an **Admin**, I want to manage user roles.
-

7. Project Architecture (Front End → Backend)

Using a single custom **API Gateway** service



High-Level Flow

Angular UI

↓ (HTTP + JWT)

API Gateway (Optional)

↓

| Identity Service |

| Customer Service |

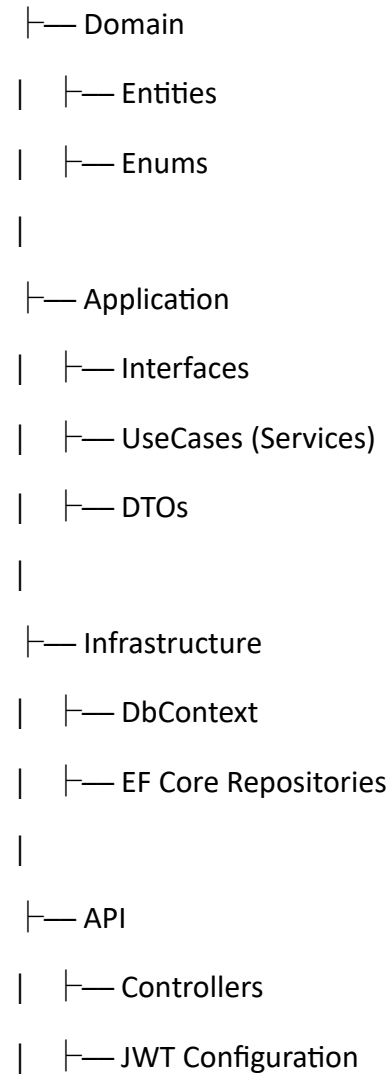
Loan Service

↓

SQL Server (EF Core Code First)

8. Clean Architecture (Backend – Per Microservice)

Each microservice follows **Clean Architecture**:



9. Technology Stack

Backend

- ASP.NET Core Web API (.NET 8)
- Entity Framework Core (Code First)
- SQL Server
- JWT Authentication
- Role-based Authorization

Frontend

- Angular (v17+)
 - Angular Routing
 - Angular Services (HttpClient)
 - Guards (Auth & Role-based)
-

10. Database Design (Simple)

Customer Table

- CustomerId (PK)
- FullName
- Email
- Phone

LoanApplication Table

- LoanId (PK)
- CustomerId (FK)
- LoanAmount
- PropertyAddress
- Status
- CreatedDate

User Table (Identity)

- UserId
 - Username
 - PasswordHash
 - Role
-

11. JWT Security Design

- Login returns **JWT token**
 - Token stored in Angular (localStorage)
 - Angular HttpInterceptor attaches token
 - ASP.NET Core validates token
 - Role-based authorization using [Authorize(Roles = "...")]
-

12. Angular Application Structure

src/

```
└─ app/
  │ └─ auth/
  │ │ └─ login
  │ │ └─ register
  │ └─ customer/
  │ └─ loan/
  │ └─ admin/
  │ └─ core/
  │ │ └─ guards
  │ │ └─ interceptors
  │ └─ services/
  └─ app-routing.module.ts
```

13. 5-Day (40 Hours) Execution Plan

Day 1 – Architecture & Setup (8 hrs)

- Domain understanding
- Microservice setup

- Clean Architecture folders
- SQL Server & EF Core setup

Day 2 – Identity Service (8 hrs)

- JWT authentication
- Role management
- Login & Register APIs

Day 3 – Customer & Loan Services (8 hrs)

- CRUD APIs
- Loan submission logic
- Status handling

Day 4 – Angular Frontend (8 hrs)

- Login/Register UI
- Loan apply form
- Loan list & status

Day 5 – Integration & Security (8 hrs)

- JWT integration Angular ↔ API
- Role-based guards
- Testing & cleanup

6. Frontend Technology Stack (Angular)

Core Framework

- **Angular (v17+)**
- TypeScript
- HTML5 / CSS3

Angular Modules & Features

Feature	Usage
Angular Routing	Page navigation
Reactive Forms	Loan application forms
HttpClient	API communication
Guards	Auth & role protection
Interceptors	JWT token injection
Lazy Loading	Role-based modules

UI Structure

app/

- ├─ auth (login, register)
- ├─ customer (profile, apply-loan)
- ├─ loan (loan-list, approval)
- ├─ admin (user-role)
- ├─ core (guards, interceptors)
- └─ shared

Frontend Deployment

- Hosted on **Azure App Service**
- Environment-based API URLs
- Secure API calls using JWT

7. Backend Technology Stack (ASP.NET Core)

Core Framework

- **ASP.NET Core (.NET 8)**
- RESTful APIs
- Clean Architecture

- Entity Framework Core

Data Layer

Service	Database
Identity Service	Azure Cosmos DB
Customer Service	Azure SQL Database
Loan Service	Azure SQL Database

- EF Core Code First
- Migrations
- Repository pattern

Security

- JWT Authentication
- Role-based Authorization
- Password hashing
- Token validation middleware

Azure Integrations

Azure Service	Purpose
Azure App Service	API & UI hosting
Azure Service Bus	Async messaging
Azure Blob Storage	Documents
Azure Key Vault	Secrets

Azure Service	Purpose
Azure Cosmos DB	Identity users

14. Expected Learning Outcomes

By the end of this case study, students will:

- Understand **real-world mortgage domain basics**
- Build **microservices using Clean Architecture**
- Implement **JWT security end-to-end**
- Integrate **Angular with secured APIs**
- Gain confidence in **enterprise project structure**